7

Carrier and Code Tracking

7.1 Motivation

The acquisition provides only rough estimates of the frequency and code phase parameters. The main purpose of tracking is to refine these values, keep track, and demodulate the navigation data from the specific satellite (and provide an estimate of the pseudorange). A basic demodulation scheme is shown in Figure 7.1.

The figure shows the scheme used to demodulate the input signal to obtain the navigation message. First, the input signal is multiplied with a carrier replica. This is done to wipe off the carrier wave from the signal. In the next step, the signal is multiplied with a code replica, and the output of this multiplication gives the navigation message. So the tracking module has to generate two replicas, one for the carrier and one for the code, to perfectly track and demodulate the signal of one satellite. In the following, a detailed description of the demodulation scheme is conducted.

7.2 Demodulation

Let f_{L1} and f_{L2} be the carrier frequencies of L1 and L2 for the signal transmitted from satellite k with powers P_C, P_{PL1}, and P_{PL2} for C/A or P code. The C/A code sequence is $C^k(t)$ and the P(Y) code sequence is $P^k(t)$. If the navigation data sequence is called $D^k(t)$, the total signal is given as

$$s^k(t) = \sqrt{2P_C}C^k(t)D^k(t)\cos(2\pi f_{\mathrm{L1}}t) + \sqrt{2P_{\mathrm{PL1}}}P^k(t)D^k(t)\sin(2\pi f_{\mathrm{L1}}t) + \sqrt{2P_{\mathrm{PL2}}}P^k(t)D^k(t)\sin(2\pi f_{\mathrm{L2}}t). \quad (7.1)$$

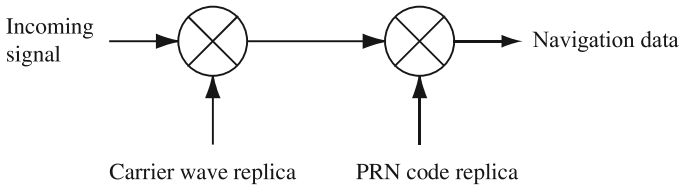


FIGURE 7.1. Basic demodulation scheme. This scheme is used to demodulate the navigation message.

The output from the front end including filtering and downconversion can be described as

$$s^k(t) = \sqrt{2P_C}C^k(t)D^k(t)\cos(\omega_{\text{IF}}t) + \sqrt{2P_{\text{PL1}}}P^k(t)D^k(t)\sin(\omega_{\text{IF}}t), \quad (7.2)$$

where ω_{IF} is the intermediate frequency to which the front end has downconverted the carrier frequency. Equation (7.2) describes the output of the front end from one satellite.

This signal is then sampled by the A/D converter. Because of the narrow bandpass filter around the C/A code, the P code is distorted. In this way the last term in Equation (7.2) is filtered out and cannot be demodulated and is in the following described as noise $e(n)$. The signal from satellite k after the A/D conversion can be described as

$$s^k(n) = C^k(n)D^k(n)\cos(\omega_{\text{IF}}n) + e(n) \quad (7.3)$$

with n in units of $1/f_s$ s; n indicates that the signal is discrete in time.

To obtain the navigation data $D^k(n)$ from the above signal, the signal has to be converted down to baseband. The carrier removal is done by multiplying the input signal with a replica of the carrier as shown in Figure 7.1. If the carrier replica is identical to the incoming carrier in both frequency and phase, the product of both is

$$\begin{aligned} s^k(n)\cos(\omega_{\text{IF}}n) &= C^k(n)D^k(n)\cos(\omega_{\text{IF}}n)\cos(\omega_{\text{IF}}n) \\ &= -\tfrac{1}{2}C^k(n)D^k(n) - \tfrac{1}{2}\cos(2\omega_{\text{IF}}n)C^k(n)D^k(n), \end{aligned} \quad (7.4)$$

where the first term is the navigation message multiplied with the PRN code and the second term is a carrier with the double intermediate frequency. The latter part of the signal can be removed by applying a lowpass filter. The signal after the lowpass filter is

$$\tfrac{1}{2}C^k(n)D^k(n). \quad (7.5)$$

The next step is to remove the code $C^k(n)$ from the signal. This is done by correlating the signal with a local code replica. If the code replica is exactly the same as the code in the signal, the output of the correlation is

$$\sum_{n=0}^{N-1} C^k(n)C^k(n)D^k(n) = ND^k(n), \quad (7.6)$$

where $ND^k(n)$ is the navigation message multiplied by the amount of points in the signal N.

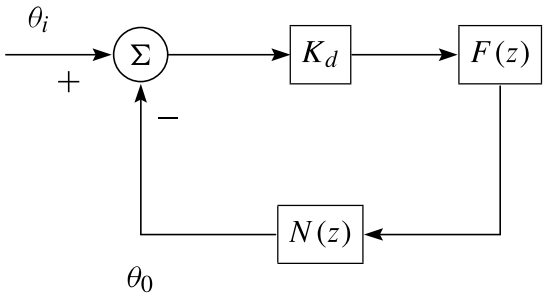


FIGURE 7.2. Linearized digital second-order PLL model.

The above description of the demodulation is only for a signal with one satellite. This is done to reduce the complexity of the equations and to give a simpler idea of the demodulation scheme. In the real signal there is a signal contribution from each visible satellite resulting in larger noise terms in the equations; see Haykin (2000), page 95.

In the demodulation scheme seen in Figure 7.1, two local signal replicas are required. To produce the exact replica some kind of feedback is needed. The feedback loop to produce the carrier replica is referred to as the carrier tracking loop, and the feedback loop to produce the exact code replica is referred to as the code tracking loop.

7.3 Second-Order PLL

Both the carrier tracking (Costas loop) and code tracking [delay lock loop (DLL)] have an analytic linear phase lock loop model that can be used to predict performance. This linear model has been derived by Ziemer & Peterson (1985) and is an extremely powerful tool to predict the performance of the tracking loop. Another excellent reference, once the fundamental models for Costas and DLL have been derived, for linear phase lock loop and its parameters and performance is by Best (2003).

Extending the linear PLL model has been derived by Chung et al. (1993). This approach will be followed by the implementation of both the Costas loop and DLL, yet the linear model references earlier can still be the basis of performance prediction and analysis.

The second-order PLL system contains a first-order filter and a voltage controlled oscillator (VCO). Note that the transfer function of an analog loop filter and a VCO are

$$F(s) = \frac{1}{s}\frac{\tau_2 s + 1}{\tau_1}, \tag{7.7}$$

$$N(s) = \frac{K_o}{s}, \tag{7.8}$$

where $F(s)$ and $N(s)$ are the transfer functions of the filter and NCO, respectively. K_o is the NCO gain. The transfer function of a linearized analog PLL is [refer to

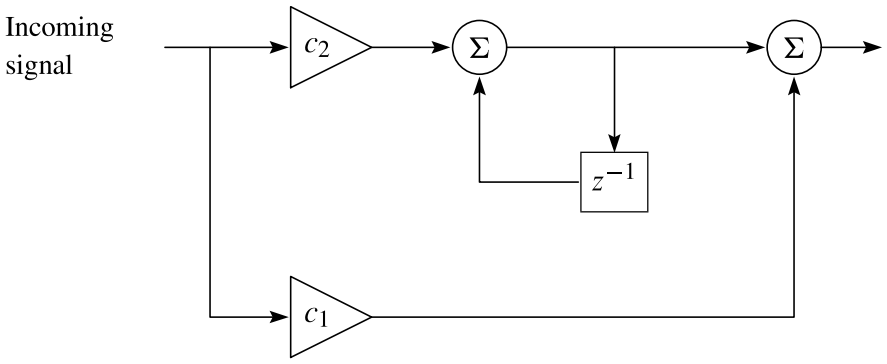


FIGURE 7.3. Second-order phase lock loop filter.

Chung et al. (1993)]

$$H(s) = \frac{K_d F(s) N(s)}{1 + K_d F(s) N(s)}, \tag{7.9}$$

where K_d is the gain of the phase discriminator. Substituting Equations (7.7) and (7.8) into the transfer function (7.9) yields

$$H(s) = \frac{2\zeta\omega_n s + \omega_n^2}{s^2 + s\zeta\omega_n s + \omega_n^2}, \tag{7.10}$$

where the natural frequency $\omega_n = \sqrt{(K_o K_d)/\tau_1}$, and the damping ratio $\zeta = (\tau_2\omega_n)/2$. The above transfer functions are analog versions and to convert the transfer functions to digital form, the bilinear transformation is used on (7.10). This yields the following digital transfer functions for the PLL model:

$$H_1(z) = \frac{\left(4\zeta\omega_n T + (\omega_n T)^2\right) + 2(\omega_n T)^2 z^{-1} + \left((\omega_n T)^2 - 4\zeta\omega_n T\right) z^{-2}}{\left(4 + 4\zeta\omega_n T + (\omega_n T)^2\right) + \left(2(\omega_n T)^2 - 8\right) z^{-1} + \left(4 - 4\zeta\omega_n T + (\omega_n T)^2\right) z^{-2}}. \tag{7.11}$$

The linearized digital second-order PLL model is shown in Figure 7.2, where K_d is the discriminator gain, $F(z)$ is the transfer function of the filter, and $N(z)$ is the transfer function of NCO. The transfer functions for the digital filter and NCO are

$$F(z) = \frac{(C_1 + C_2) - C_1 z^{-1}}{1 - z^{-1}}, \tag{7.12}$$

$$N(z) = \frac{K_o z^{-1}}{1 - z - 1}, \tag{7.13}$$

where $F(z)$ is the transfer function of the filter and $N(z)$ is the transfer function of the NCO. Figure 7.3 shows the phase lock filter $F(z)$.

The goal is to find the coefficients C_1 and C_2 in the second-order PLL. This is done by comparing the transfer function for the digital PLL and the transfer function for the analog PLL. The transfer function for the digital version can be found as

$$H(z) = \frac{\theta_o(z)}{\theta_i(z)} = \frac{K_d F(z) N(z)}{1 + K_d F(z) N(z)}. \tag{7.14}$$

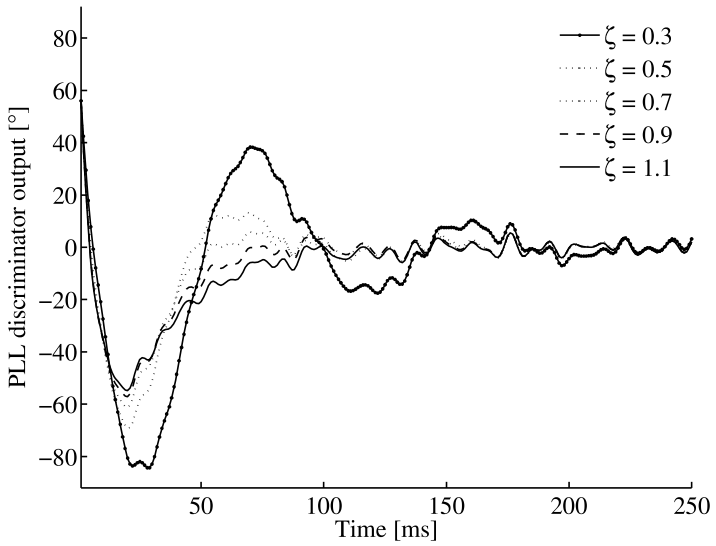


FIGURE 7.4. Phase error as function of different damping ratios ζ. A larger settling time results in a smaller overshoot of the phase.

By substituting (7.12) and (7.13) into (7.14), we obtain the following:

$$H_2(z) = \frac{K_o K_d (C_1 + C_2) z^{-1} - K_o K_d C_1 z^{-2}}{1 + \big(K_o K_d (C_1 + C_2) - 2\big) z^{-1} + (1 - K_o K_d C_1) z^{-2}}. \tag{7.15}$$

To find an equation for the two coefficients C_1 and C_2, (7.11) and (7.15) are compared. This yields the following two equations:

$$C_1 = \frac{1}{K_o K_d} \frac{8\zeta\omega_n T}{4 + 4\zeta\omega_n T + (\omega_n T)^2}, \tag{7.16}$$

$$C_2 = \frac{1}{K_o K_d} \frac{4(\omega_n T)^2}{4 + 4\zeta\omega_n T + (\omega_n T)^2}, \tag{7.17}$$

where $K_o K_d$ is the loop gain, ζ is the damping ratio, ω_n is the natural frequency, and T is the sampling time; see Chung et al. (1993).

The natural frequency can be found as

$$\omega_n = \frac{8\zeta B_L}{4\zeta^2 + 1}, \tag{7.18}$$

where B_L is the noise bandwidth in the loop; see Parkinson & Spilker Jr. (1996), volume 1, page 371.

The damping ratio and noise bandwidth are computed for a particular signal case. But in some cases an engineer would like to change these values for specific applications or implementations. Therefore, a more thorough explanation is given about these parameters.

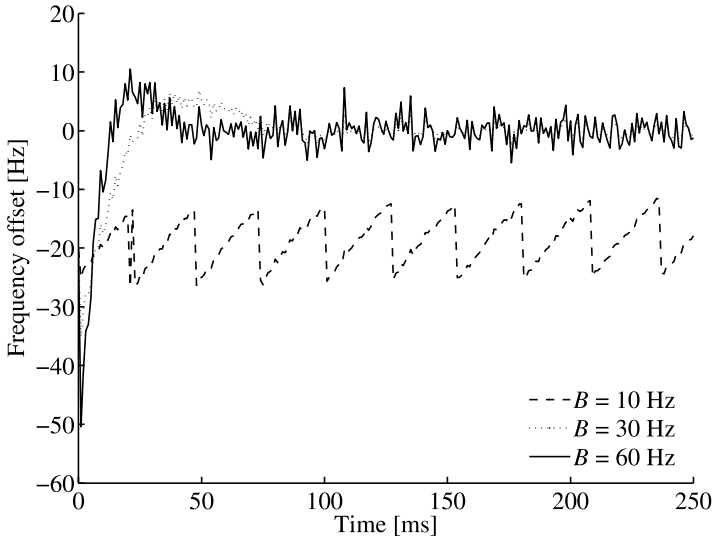


FIGURE 7.5. Frequency offsets from an acquired frequency offset 20 Hz and for PLL noise bandwidths of 10, 30, and 60 Hz. There are negative peaks in the first 2 ms due to transition phase in the loop filter.

7.3.1 Damping Ratio

The damping ratio controls how fast the filter reaches its settle point. The damping ratio also controls how much overshoot the filter can have. A smaller settling time results in a larger overshoot. This can be seen in Figure 7.4.

The choice of damping ratio is a compromise between overshoot and settling time. The damping is chosen to $\zeta = 0.7$ resulting in a filter that converges reasonably fast and does not make a high overshoot.

7.3.2 Noise Bandwidth

The second parameter in the PLL filter is the noise bandwidth B_L. The noise bandwidth controls the amount of noise allowed in the filter. This parameter can also, as the damping ratio, control the settling time. As the tracking loop starts to track a signal the start frequency is the frequency found by the acquisition algorithm. (This phase is sometimes called the pull-in phase in the literature. It is in this phase the filter is trying to converge to the correct frequency and phase.) The start frequency from the acquisition algorithm can be off by some Hz. The tracking loop is then going to lock onto the correct frequency. To see the impact of various noise bandwidths, a real GPS signal is used where the acquisition algorithm found a frequency that is about 21 Hz off. Figure 7.5 shows the offset from the start frequency for three different noise bandwidths.

From the figure, it follows that if the noise bandwidth is 60 Hz, the tracking loop immediately finds the correct frequency offset of about 21 Hz. It can also be seen that a lot of noise in the tracking frequency is allowed. In the second case, where the noise bandwidth is 30 Hz, the tracking loop also locks on the signal

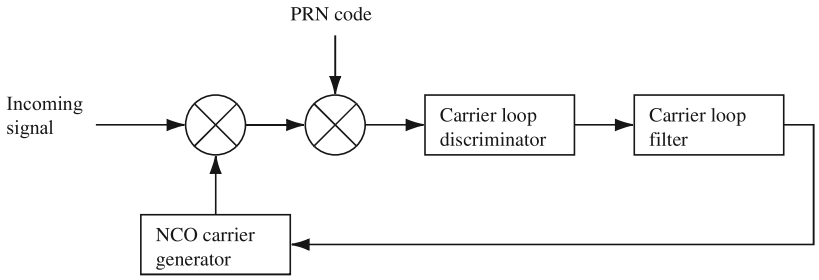


FIGURE 7.6. Basic GPS receiver tracking loop block diagram.

quite fast. It can be seen that the noise on the tracking frequency is much smaller than with a noise bandwidth of 60 Hz. In the third case where the noise bandwidth is 10 Hz, the tracking loop is not fast enough to reach the real frequency before a phase shift occurs. Therefore, it is not likely to converge to the proper value. A large noise bandwidth implies that the tracking loop quickly locks to the real frequency but has a relatively large frequency noise in the locked state. A smaller noise bandwidth implies that it can take some time before the tracking loop can be locked to the frequency, but after the lock the frequency is stable. Some implementations split the PLL into two filters, often called pull-in and tracking filters.

For land applications, a typical value for noise bandwidth is about 20 Hz.

7.4 Carrier Tracking

To demodulate the navigation data successfully an exact carrier wave replica has to be generated. To track a carrier wave signal, phase lock loops (PLL) or frequency lock loops (FLL) are often used.

Figure 7.6 shows a basic block diagram for a phase lock loop. The two first multiplications wipe off the carrier and the PRN code of the input signal. To wipe off the PRN code, the I_p output from the early–late code tracking loop described above is used. The loop discriminator block is used to find the phase error on the local carrier wave replica. The output of the discriminator, which is the phase error (or a function of the phase error), is then filtered and used as a feedback to the numerically controlled oscillator (NCO), which adjusts the frequency of the local carrier wave. In this way the local carrier wave could be an almost precise replica of the input signal carrier wave.

The problem with using an ordinary PLL is that it is sensitive to 180° phase shifts. Due to navigation bit transitions, a PLL used in a GPS receiver has to be insensitive to 180° phase shifts.

Figure 7.7 shows a Costas loop. One property of this loop is that it is insensitive for 180° phase shifts and hereby a Costas loop is insensitive for phase transitions due to navigation bits. This is the reason for using this carrier tracking loop in GPS receivers. The Costas loop in Figure 7.7 contains two multiplications. The first multiplication is the product between the input signal and the local carrier wave and the second multiplication is between a 90° phase-shifted carrier wave

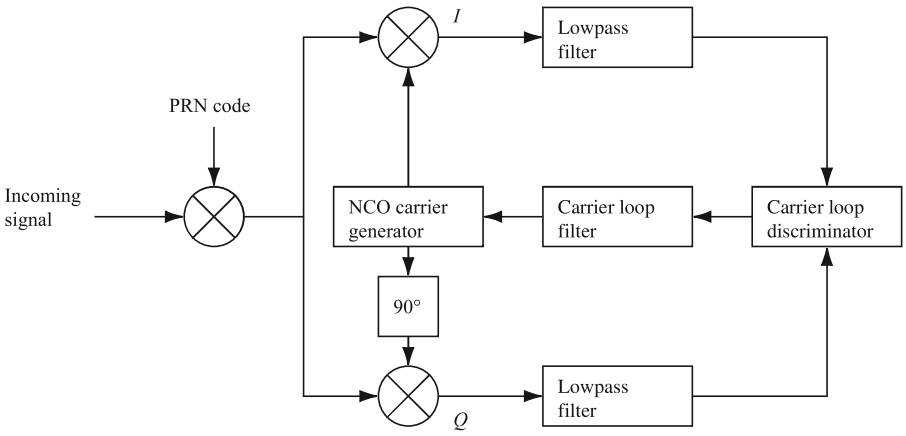


FIGURE 7.7. Costas loop used to track the carrier wave.

and the input signal. The *goal of the Costas loop is to try to keep all energy in the I (in-phase) arm*. To keep the energy in the I arm, some kind of feedback to the oscillator is needed. If it is assumed that the code replica in Figure 7.7 is perfectly aligned, the multiplication in the I arm yields the following sum:

$$D^k(n)\cos(\omega_{\mathrm{IF}}\, n)\cos(\omega_{\mathrm{IF}}\, n+\varphi) = \tfrac{1}{2}D^k(n)\cos(\varphi) + \tfrac{1}{2}D^k(n)\cos(2\omega_{\mathrm{IF}}\, n+\varphi), \tag{7.19}$$

where φ is the phase difference between the phase of the input signal and the phase of the local replica of the carrier phase. The multiplication in the quadrature arm gives the following:

$$D^k(n)\cos(\omega_{\mathrm{IF}}\, n)\sin(\omega_{\mathrm{IF}}\, n+\varphi) = \tfrac{1}{2}D^k(n)\sin(\varphi) + \tfrac{1}{2}D^k(n)\sin(2\omega_{\mathrm{IF}}\, n+\varphi). \tag{7.20}$$

If the two signals are lowpass filtered after the multiplication, the two terms with the double intermediate frequency are eliminated and the following two signals remain:

$$I^k = \tfrac{1}{2}D^k(n)\cos(\varphi), \tag{7.21}$$

$$Q^k = \tfrac{1}{2}D^k(n)\sin(\varphi). \tag{7.22}$$

To find a term to feed back to the carrier phase oscillator, it can be seen that the phase error of the local carrier phase replica can be found as

$$\frac{Q^k}{I^k} = \frac{\frac{1}{2}D^k(n)\sin(\varphi)}{\frac{1}{2}D^k(n)\cos(\varphi)} = \tan(\varphi), \tag{7.23}$$

$$\varphi = \tan^{-1}\left(\frac{Q^k}{I^k}\right). \tag{7.24}$$

From Equation (7.24), it can be seen that the phase error is minimized when the correlation in the quadrature-phase arm is zero and the correlation value in the in-phase arm is maximum. The arctan discriminator in Equation (7.24) is the most precise of the Costas discriminators, but it is also the most time-consuming. Table 7.1 describes other possible Costas discriminators.

TABLE 7.1. Various types of Costas phase lock loop discriminators

Discriminator	**Description**
$D = \text{sign}(I^k)Q^k$	The output of the discriminator is proportional to $\sin(\varphi)$.
$D = I^k Q^k$	The discriminator output is proportional to $\sin(2\varphi)$.
$D = \tan^{-1}\left(\frac{Q^k}{I^k}\right)$	The discriminator output is the phase error.

Figure 7.8 shows the responses corresponding to the different discriminators. The phase discriminator outputs in this figure are computed using expressions in Table 7.1 for all possible phase errors. In the same figure it can be seen that the discriminator outputs are zero when the real phase error is 0 and $\pm 180°$. This is why the Costas loop is insensitive to the 180° phase shifts in case of a navigation bit transition.

The behavior of the Costas loop when a 180° phase shift occurs is more clearly illustrated in Figure 7.9. In this figure the vector sum of I^k and Q^k is shown as the vector in the coordinate system. If the local carrier wave were in phase with the input signal, the vector would be aligned to the I-axis, but in the present case a small phase error is illustrated. When the signal is tracked correctly the vector sum of I^k and Q^k tends to remain aligned with the I-axis. This property ensures that if a navigation bit transition occurs, the vector on the phasor diagram will flip 180° (showed by the dashed vector in the figure). If a navigation bit transition occurs, the Costas loop will still track the signal and nothing will happen. This property does make Costas loop the commonly chosen phase lock loop in GPS receivers; see Kaplan & Hegarty (2006), pages 166–170.

The output of the phase discriminator is filtered to predict and estimate any relative motion of the satellite and to estimate the Doppler frequency.

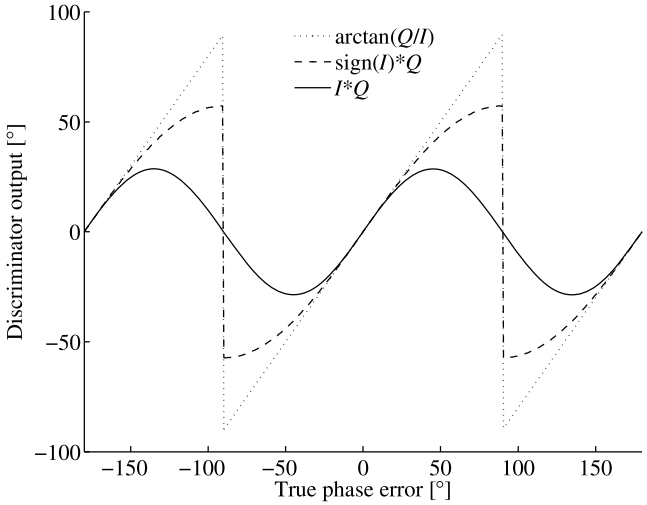


FIGURE 7.8. Comparison between the common Costas phase lock loop discriminator responses.

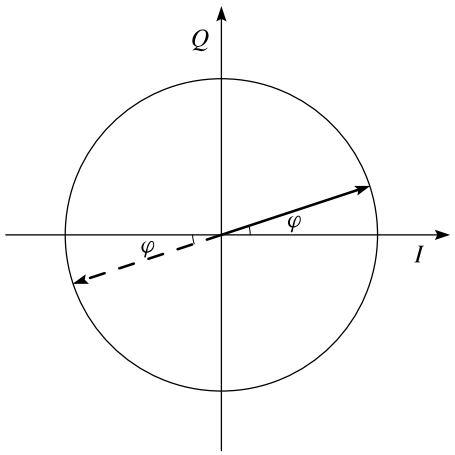


FIGURE 7.9. Phasor diagram showing the phase error between the phase of the input carrier wave and the phase of the local carrier wave replica.

7.5 Code Tracking

The goal for a code tracking loop is to keep track of the code phase of a specific code in the signal. The output of such a code tracking loop is a perfectly aligned replica of the code. The code tracking loop in the GPS receiver is a delay lock loop (DLL) called an early–late tracking loop. The idea behind the DLL is to correlate the input signal with three replicas of the code seen in Figure 7.10.

The first step in Figure 7.10 is converting the C/A code to baseband, by multiplying the incoming signal with a perfectly aligned local replica of the carrier wave. Afterwards the signal is multiplied with three code replicas. The three replicas are nominally generated with a spacing of $\pm\frac{1}{2}$ chip. After this second multiplication, the three outputs are integrated and dumped. The output of these integrations is a numerical value indicating how much the specific code replica correlates with the code in the incoming signal.

The three correlation outputs I_E, I_P, and I_L are then compared to see which one provides the highest correlation. Figure 7.11 shows an example of code tracking. In Figure 7.11a the late code has the highest correlation, so the code phase must

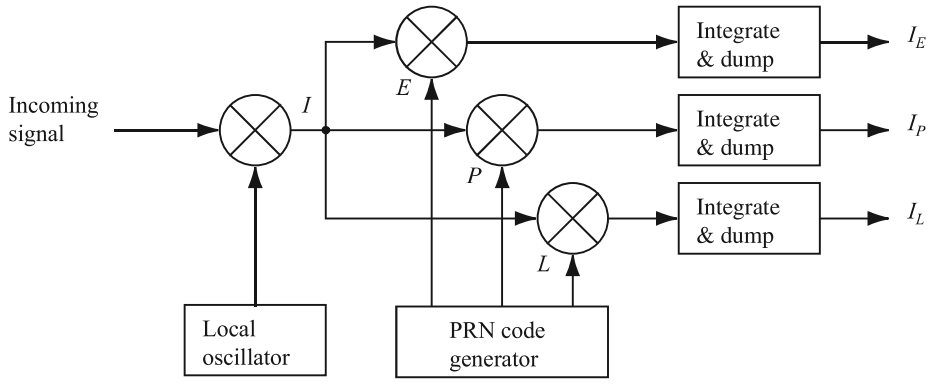


FIGURE 7.10. Basic code tracking loop block diagram.

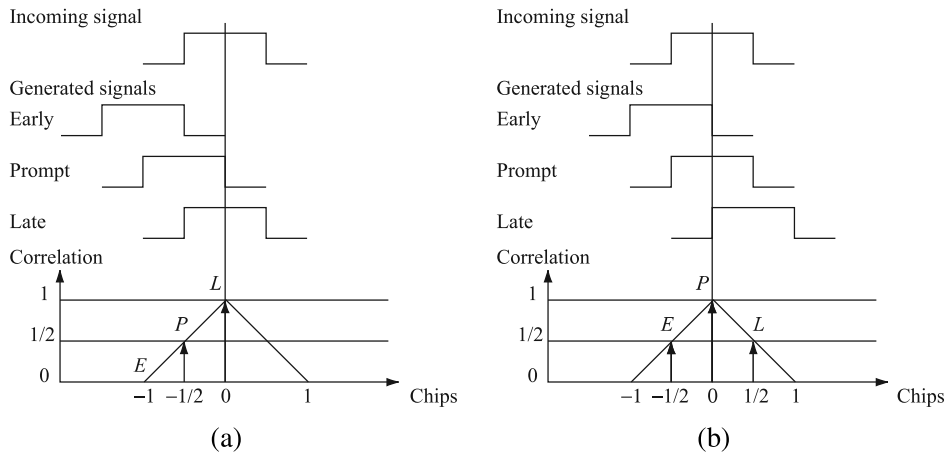


FIGURE 7.11. Code tracking. Three local codes are generated and correlated with the incoming signal. (a) The late replica has the highest correlation so the code phase must be decreased, i.e., the code sequence must be delayed. (b) The prompt code has the highest correlation, and the early and late have similar correlation. The loop is perfectly tuned in.

be decreased. In Figure 7.11b the highest peak is located at the prompt replica, and the early and late replicas have equal correlation. In this case, the code phase is properly tracked.

The DLL with three correlators as in Figure 7.10 is optimal when the local carrier wave is locked in phase and frequency. But when there is a phase error on the local carrier wave, the signal will be more noisy, making it more difficult for the DLL to keep lock on the code. So instead the DLL in a GPS receiver is often designed as in Figure 7.12.

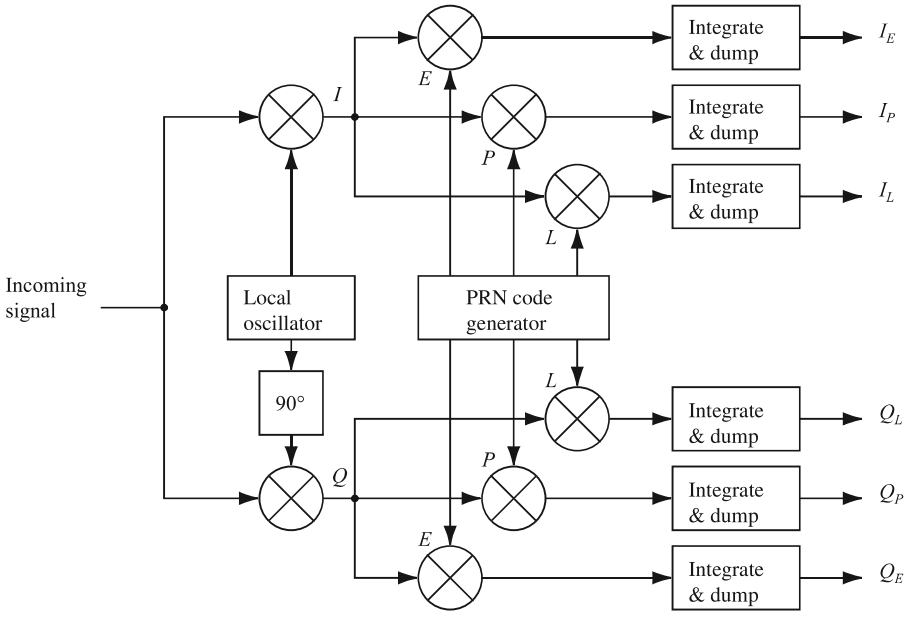


FIGURE 7.12. DLL block diagram with six correlators.

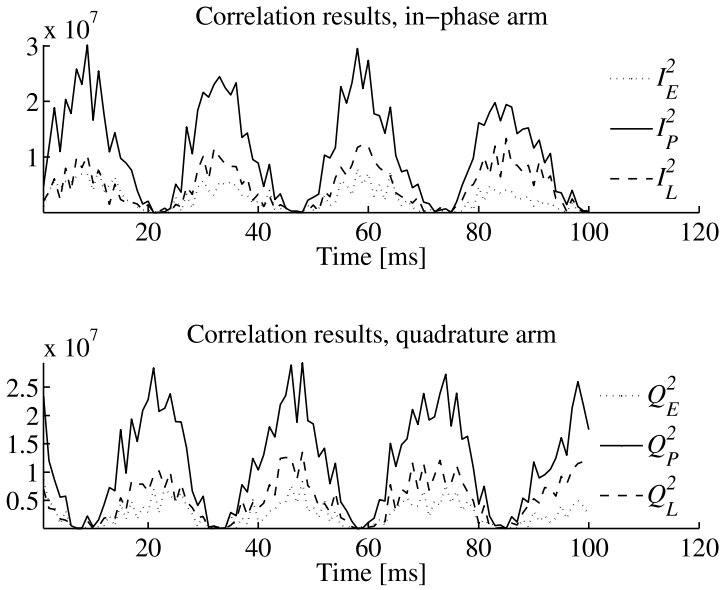


FIGURE 7.13. Output of the six correlators in the in-phase and quadrature arms of the tracking loop. Acquisition frequency offset is 20 Hz and PLL noise bandwidth is 15 Hz (for demonstration purpose).

The design in Figure 7.12 has the advantage that it is independent of the phase on the local carrier wave. If the local carrier wave is in phase with the input signal, all the energy will be in the in-phase arm. But if the local carrier phase drifts compared to the input signal, the energy will switch between the in-phase and the quadrature arm. For demonstration purposes, Figure 7.13 shows such a situation

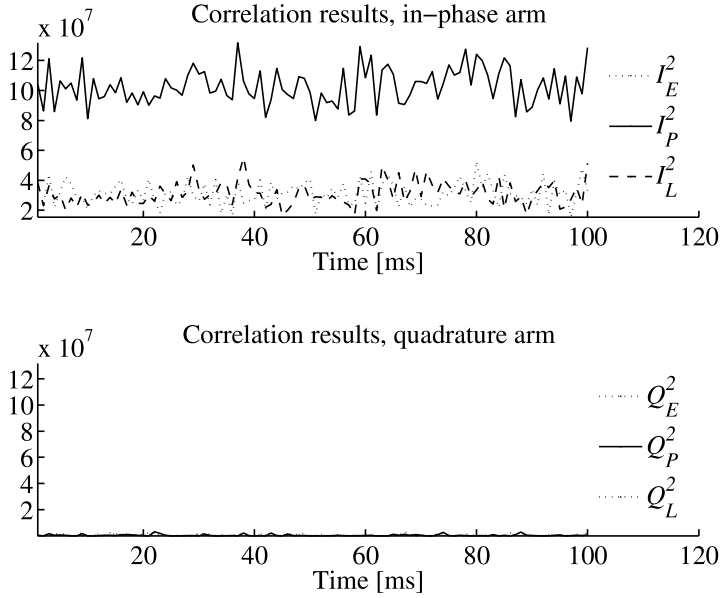


FIGURE 7.14. Output of the six correlators in the in-phase and quadrature arms of the tracking loop. The local carrier wave is in phase with the input signal.

TABLE 7.2. Various types of delay lock loop discriminators and a description of them

Type	Discriminator D	Characteristics
Coherent	$I_E - I_L$	Simplest of all discriminators. Does not require the Q branch but requires a good carrier tracking loop for optimal functionality.
Noncoherent	$(I_E^2 + Q_E^2) - (I_L^2 + Q_L^2)$	Early minus late power. The discriminator response is nearly the same as the coherent discriminator inside $\pm\frac{1}{2}$ chip.
	$\dfrac{(I_E^2 + Q_E^2) - (I_L^2 + Q_L^2)}{(I_E^2 + Q_E^2) + (I_L^2 + Q_L^2)}$	Normalized early minus late power. The discriminator has a great property when the chip error is larger than a $\frac{1}{2}$ chip; this will help the DLL to keep track in noisy signals.
	$I_P(I_E - I_L) + Q_P(Q_E - Q_L)$	Dot product. This is the only DLL discriminator that uses all six correlator outputs.

where the phase of the carrier replica drifts compared to the phase of the incoming signal. The upper plot shows the output of the three correlators in the in-phase arm, and the lower plot shows the correlation output in the quadrature arm of the DLL with six correlators. This situation is a result of different frequencies for the signal and the replica; it results in a constantly changing phase difference (misalignment). There are a few reasons why this can happen, for example, the PLL could be not in a lock state.

Figure 7.14 shows a case when the PLL is in a lock state. Because of the precise carrier replica from the PLL, it is seen in Figure 7.14 that the correlators are constant over time. This would not be the case if the carrier replica is not adjusted to match the frequency and phase of the incoming signal.

If the code tracking loop performance has to be independent of the performance of the phase lock loop, the tracking loop has to use both the in-phase and quadrature arms to track the code.

The DLL now needs a feedback to the PRN code generators if the code phase has to be adjusted. Some common DLL discriminators used for feedback are listed in Table 7.2.

The table shows one coherent and three noncoherent discriminators. The requirements of a DLL discriminator is dependent on the type of application and the noise in the signal. The discriminator function responses are shown in Figure 7.15.

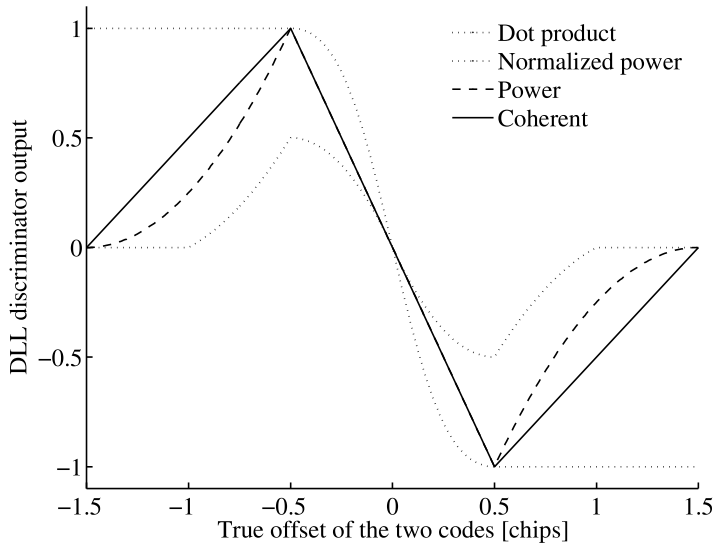


FIGURE 7.15. Comparison between the common DLL discriminator responses.

Figure 7.15 shows the coherent discriminator and three noncoherent discriminators using a standard correlator. The figure is produced from ideal ACFs, and the space between the early, prompt, and late is $\pm\frac{1}{2}$ chip. The space between the early, prompt, and late codes determines the noise bandwidth in the delay lock loop. If the discriminator spacing is larger than $\frac{1}{2}$ chip, the DLL would be able to handle wider dynamics and be more noise robust; on the other hand, a DLL with a smaller spacing would be more precise. In a modern GPS receiver the discriminator spacing can be adjusted while the receiver is tracking the signal. The advantage from this is that if the signal-to-noise ratio suddenly decreases, the receiver uses a wider spacing in the correlators to be able to handle a more noisy signal, and hereby a possible code lock loss could be avoided; see Kaplan & Hegarty (2006), page 175.

The implemented tracking loop discriminator is the normalized early minus late power. This discriminator is described as

$$D = \frac{(I_E^2 + Q_E^2) - (I_L^2 + Q_L^2)}{(I_E^2 + Q_E^2) + (I_L^2 + Q_L^2)}, \tag{7.25}$$

where I_E, Q_E, I_L, and Q_L are output from four of the six correlators shown in Figure 7.13. The normalized early minus late power discriminator is chosen because it is independent of the performance of the PLL as it uses both the in-phase and quadrature arms. The normalization of the discriminator causes that the discriminator can be used with signals with different signal-to-noise ratios and different signal strengths.

The tracking loop generates three local code replicas. In this section, the chip space between the early and prompt replicas is half a chip.

As was described, the DLL can be modeled as a linear PLL and thus the performance of the loop can be predicted based on this model. In other words the loop filter design is the same, just the parameter values are different.

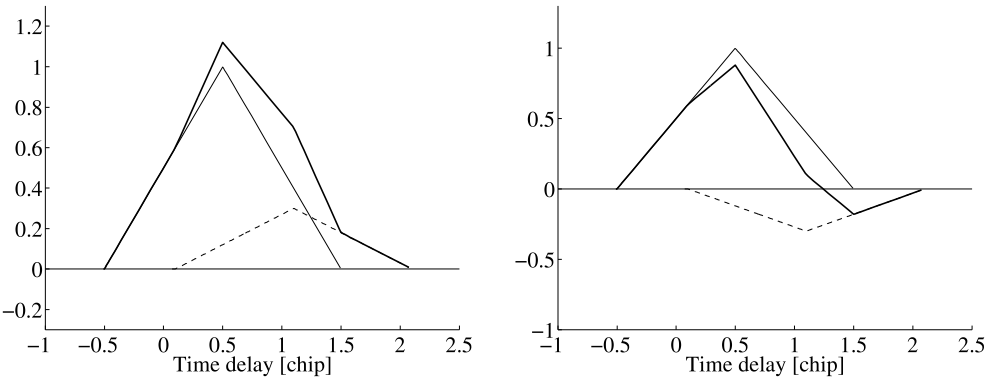


FIGURE 7.16. Constructive and destructive multipath interference. The solid thin line represents the original triangular correlation function; the dashed line the multipath correlation function, and the third line is the sum of the two.

7.6 Multipath

Of the multiple error sources associated with GNSS signal processing, multipath directly impacts the code tracking performance. With the prior description of how code tracking is implemented, it makes sense to investigate how multipath impacts the code tracking loop.

The signal observed at the receiver is a distorted version of the one transmitted. One distortion effect is called multipath propagation or, for short, *multipath*.

If the receiver can directly see the satellite, a part of the received signal has propagated via the direct path from the satellite to the receiver. This signal component is delayed according to the distance between the satellite transmitter and the receiver. The signal component propagated via the direct path usually is by far the strongest part of the received signal.

In addition to the direct signal, the receiver may observe other signals propagating via other and longer paths. This can happen if the radio wave reaches the receiver after interaction with one or more objects/obstacles in the environment. Different kinds of interactions between radio waves and objects exist, but in the context of GNSS it suffices to think of an interaction as a reflection altering the direction of propagation, amplitude, polarity, and phase of the radio wave.

First we consider the simple case where the transmitted signal reaches the receiver via two paths. Thus, the received signal consists of two components: a direct component and a signal component reaching the receiver via a reflection on a nearby building. In this case the reflected signal component is a delayed, phase-shifted, and attenuated version of the line-of-sight signal component. Due to the change in phase and delay, the two signal components interfere. In case the reflected and the direct signals are in phase, the amplitude of the sum is larger than the amplitude of each of the components. This we call *constructive interference*; see Figure 7.16. On the other hand, if the direct and reflected signals are out of phase, the amplitude of the sum signal decreases and we talk about *destructive interference*. In case the relative phase between the direct and reflected signals

rapidly changes the amplitude of the sum signal, we say that the received signal is *fading*. A signal component reaching the receiver via a different path than the direct path is called a *multipath component*.

In general, the received signal $x(t)$ is composed of the direct signal and $M-1$ multipath components. Let $A_i(t)$ denote the amplitude of the ith multipath component, let D denote the navigation message, let C denote the code, let τ denote the multipath error, let the frequency change be v_i, the phase offset be φ_i, and, finally, add a noise term $n(t)$; then the signal can be described as

$$x(t) = \sum_{i=1}^{M} A_i(t) D\big(t-\tau_i(t)\big) C\big(t-\tau_i(t)\big) \cos\big(2\pi(f_0+v_i(t))t+\varphi_i(t)\big) + n(t).$$

To simplify the discussion we consider a two-path scenario ($M=2$) and make the following assumptions:

$$\begin{aligned} A_1(t) &= A_1; & A_2(t) &= A_2;\\ \tau_1(t) &= \tau_1; & \tau_2(t) &= \tau_2;\\ v_1(t) &= v_1; & v_2(t) &= v_2;\\ n(t) &= 0. \end{aligned}$$

In other words we have assumed that the parameters (amplitudes, delays, and Doppler shifts) are constant over the time period we consider. In this case, $x(t)$ can be reduced to

$$\begin{aligned} x(t) = {} & A_1 D(t-\tau_1) C(t-\tau_1) \cos\big(2\pi(f_0+v_1)t+\varphi_1\big)\\ & + A_2 D(t-\tau_2) C(t-\tau_2) \cos\big(2\pi(f_0+v_2)t+\varphi_2\big). \end{aligned} \tag{7.26}$$

We now exploit $\cos(\varphi) = \Re\big(\exp(j\varphi)\big)$ to rewrite (7.26) as

$$\begin{aligned} x(t) &= \Re\Big\{ A_1 D(t-\tau_1) C(t-\tau_1) \exp\big[j\big(2\pi(f_0+v_1)t+\varphi_1\big)\big]\\ &\qquad + A_2 D(t-\tau_2) C(t-\tau_2) \exp\big[j\big(2\pi(f_0+v_2)t+\varphi_2\big)\big]\Big\}\\ &= \Re\Big\{ A_1 D(t-\tau_1) C(t-\tau_1) \exp\big[j\big(2\pi(f_0+v_1)t+\varphi_1\big)\big]\\ &\qquad + A_2 D(t-\tau_2) C(t-\tau_2) \exp\big[j\big(2\pi(v_2-v_1)t+(\varphi_2-\varphi_1)\big)\big]\\ &\qquad\qquad \times \exp\big[j\big(2\pi(f_0+v_1)t+\varphi_1\big)\big]\Big\}\\ &= \Re\Big\{\Big(A_1 D(t-\tau_1) C(t-\tau_1)\\ &\qquad + A_2 D(t-\tau_2) C(t-\tau_2) \exp\big[j\big(2\pi(v_2-v_1)t+(\varphi_2-\varphi_1)\big)\big]\Big)\\ &\qquad\qquad \times \exp\big[j\big(2\pi(f_0+v_1)t+\varphi_1\big)\big]\Big\}. \end{aligned}$$

Defining the instantaneous phase difference between the two signal components $\psi(t) = 2\pi(v_2-v_1)t + (\varphi_2-\varphi_1)$, the output of the integrators in the DLL can be

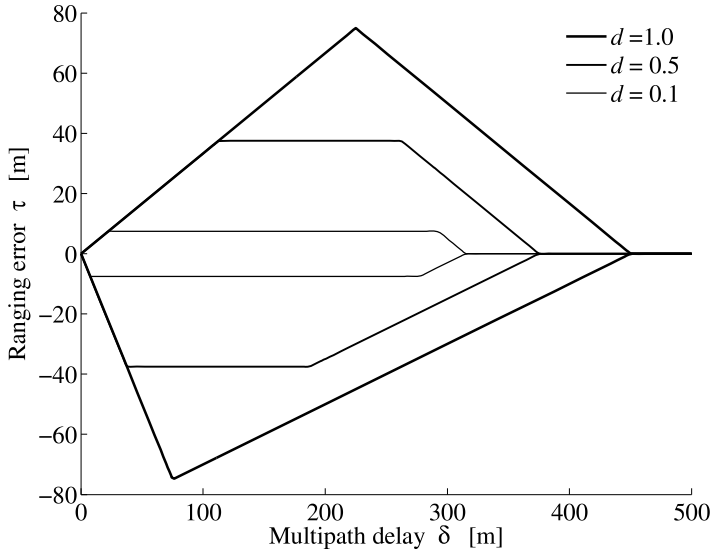


FIGURE 7.17. Multipath error envelope for noncoherent early/late detector for C/A code, $\alpha = 0.5$. The positive multipath error corresponds to constructive interference while negative multipath error corresponds to destructive interference.

approximated as

$$\begin{aligned} y_{\pm}(t) &\approx A_1 R_C(\tau_1 - \hat{\tau} \pm \delta) D(t - \tau_1) + A_2 R_C(\tau_2 - \hat{\tau} \pm \delta) \exp\big(j\psi(t)\big) D(t - \tau_2) \\ &\approx \Big[A_1 R_C(\tau_1 - \hat{\tau} \pm \delta) + A_2 \exp\big(j\psi(t)\big) R_C(\tau_2 - \hat{\tau} \pm \delta)\Big] D(t - \tau_1). \end{aligned}$$

With $\epsilon = \tau_1 - \hat{\tau}$, the envelope of $y_{\pm}(t)$ is

$$|y_{\pm}(t)| = \Big|A_1 R_C(\epsilon \pm \delta) + A_2 \exp\big(j\psi(t)\big) R_C\big(\epsilon + (\tau_2 - \tau_1) \pm \delta\big)\Big|. \tag{7.27}$$

The output of the envelope discriminator is

$$\epsilon(t) = |y_-(t)| - |y_+(t)|. \tag{7.28}$$

As mentioned before, the amplitude of the sum signal depends on the relative phase $\psi(t)$ between the direct and the reflected signal components. We consider the two special cases of constructive and destructive interference:

Constructive interference In this case $\exp\big(j\psi(t)\big) = 1$ and

$$|y_{\pm}(t)| = A_1 R_C(\epsilon \pm \delta) + A_2 R_C(\epsilon + \tau_2 - \tau_1 \pm \delta). \tag{7.29}$$

Destructive interference Here, $\exp\big(j\psi(t)\big) = -1$ and, therefore,

$$|y_{\pm}(t)| = A_1 R_C(\epsilon \pm \delta) - A_2 R_C(\epsilon + \tau_2 - \tau_1 \pm \delta). \tag{7.30}$$

If the receiver is moving, it is very likely that $v_1 \neq v_2$ and the relative phase changes over time. Therefore, the envelope of the received signal will fluctuate

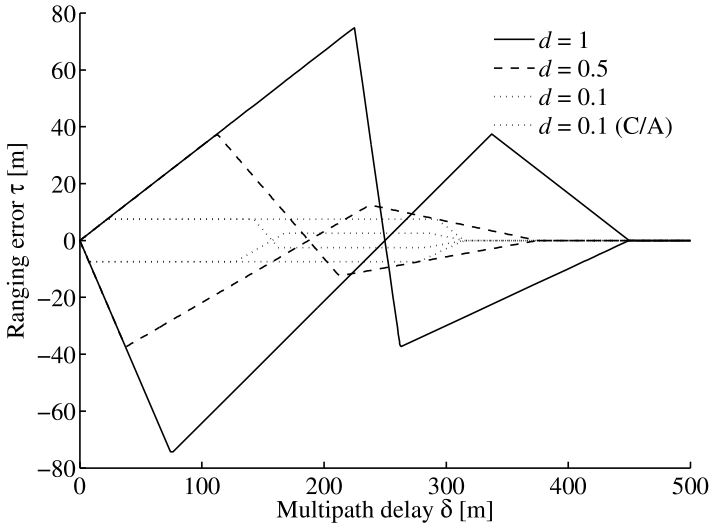


FIGURE 7.18. Multipath error envelope for noncoherent early/late detector for BOC(1,1). Negative ranging error corresponds to destructive interference.

over time as the two signal components interfere. As a measure of the magnitude of the fluctuations we define the *relative amplitude* as

$$\alpha = \frac{A_2}{A_1}.$$

In practice, the amplitude of the direct component is larger than the reflected component and thus α is somewhat less than one.

Equation (7.29) is graphed as the upper half of Figure 7.17 for $\alpha = 0.5$ and for three values of the correlator spacing $d = 0.1, 0.5$, and 1.0. The upper part of the figure corresponds to constructive interference while the lower part corresponds to destructive interference; see Equation (7.30). All actual multipath errors lie within the combined envelope!

In Figure 7.17 the initial slope is a function of multipath amplitude and delay δ only. It is independent of correlator and PRN chipping rate f_c. We recall the C/A-code pseudorange multipath error can approach 147 m, theoretically. However, errors of 10 m or less are far more common. Large errors can be encountered in urban environments.

Our investigation assumes infinite GPS signal bandwidth. Bandwidths of 10–20 MHz yield results that are similar to those for the infinite bandwidth case. For short-delay multipath (small δ), the finite bandwidth effects are much less significant.

In conclusion: Multipath propagation deforms the ideal correlation peak because the received signal is a sum of signal components. The multipath components arrive later at the receiver and contribute additional correlation peaks. Thus, the early–late correlator samples may not be centered on the true arrival time of the direct path.

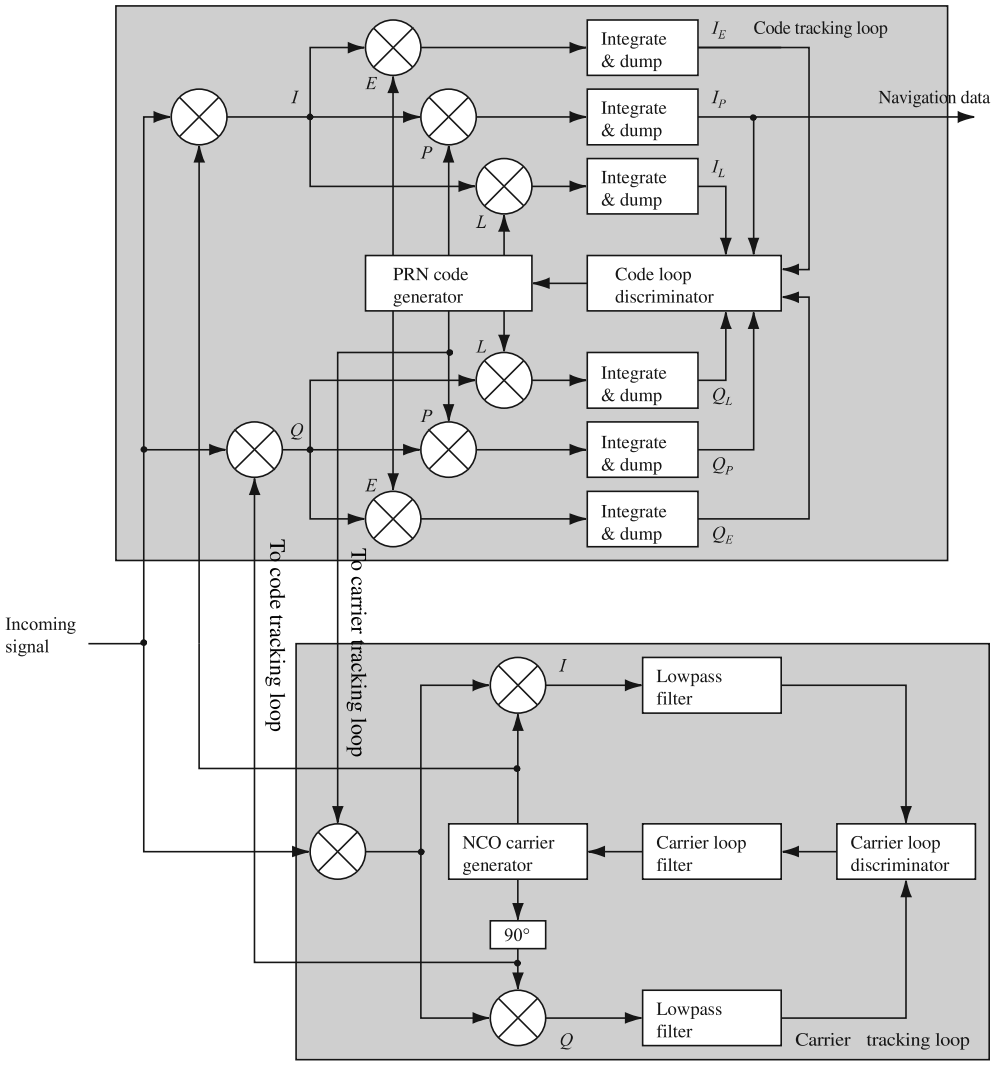


FIGURE 7.19. Block diagram of the combined DLL and PLL tracking loops.

We note that in MATLAB the ACF can be coded as

```
R = (1 - abs(tau)) * heaviside(1 - abs(tau));
```

Next we determine the multipath error envelope for Galileo BOC(1,1). The result is plotted in Figure 7.18. Comparing Figures 7.17 and 7.18 for correlator spacing $d = 0.1$ we see that the C/A code multipath error envelop is sensitive for multipath signals with a relative path delay up to 300 m. The resulting range errors τ are between ± 4 m. For BOC(1,1) the corresponding value is 150 m and range errors of ± 4 m. This demonstrates that BOC(1,1) signals are better to handle multipath signals than C/A code signals!

The multipath error envelopes are computed from equations like (7.29) and (7.30). Figures 7.17 and 7.18 are plotted by using the powerful ezplot command. With correlator spacing $d = 0.1$ and multipath delays δ in the range 0–157 m

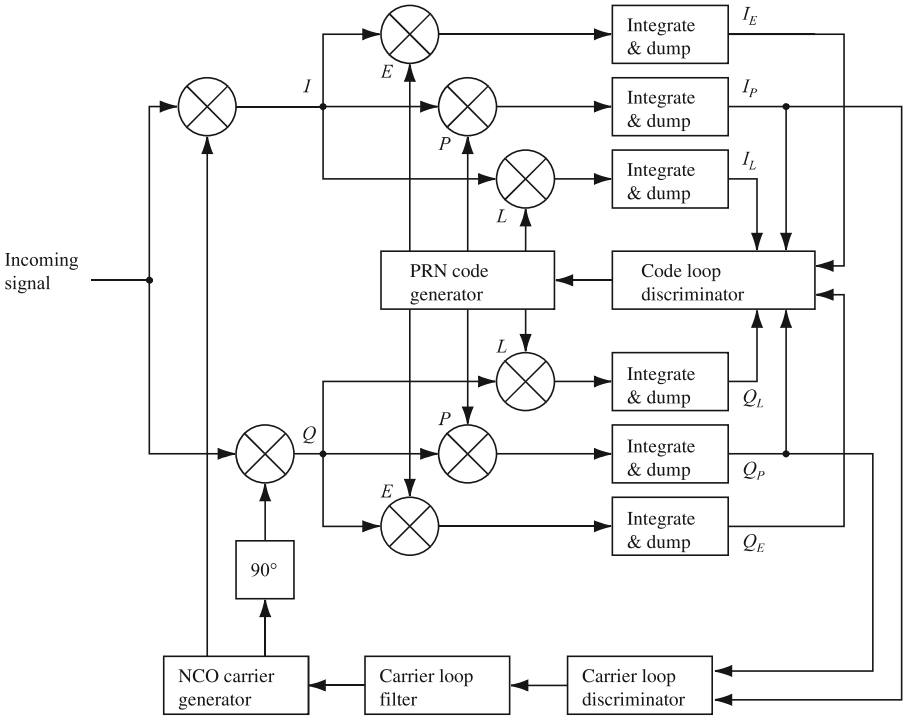


FIGURE 7.20. The block diagram of a complete tracking channel on the GPS receiver.

we obtain ranging errors τ in the interval ± 7.5 m; for δ in the range 157–317 m the delay τ is within ± 2.5 m. Obviously the delay range τ strongly depends on the correlator spacing d; some papers even mention $d = 1/25$. See also Winkel (2005).

7.7 Complete Tracking Block

In the previous sections, the code tracking loop and the carrier tracking loop are described in detail. The following describes how the code tracking loop and the carrier tracking loop can be joined to minimize the computational load.

Figure 7.19 shows the code tracking loop and the carrier tracking loop combined. It can be seen from the figure that the PRN code replica used to wipe off the PRN code in the carrier tracking loop is coming from the code tracking loop. It can also be seen that the two local carrier replicas used to wipe off the carrier wave in the code tracking loop are coming from the carrier tracking loop. The block diagram in Figure 7.19 contains 11 multiplications. These multiplications are the most time-consuming operations on the block diagram.

Figure 7.20 shows an optimized version of the combined tracking loops. Here the I and Q inputs to the phase discriminator are the I_p and Q_p correlation from the code tracking loop. In this way the three multiplications in the Costas loop are eliminated, and hereby the computation time is reduced.

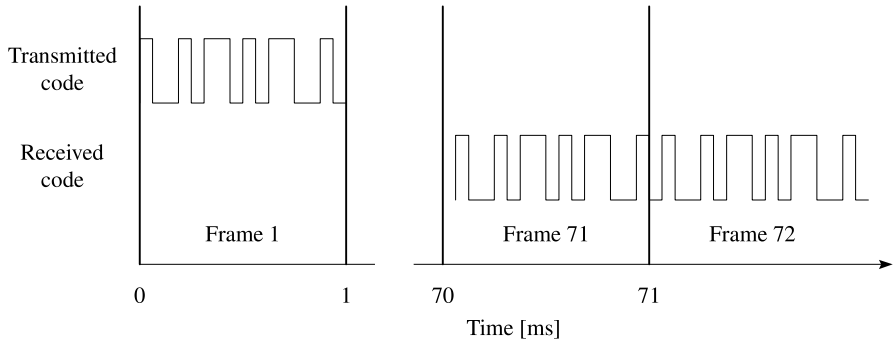


FIGURE 7.21. The delay between the time of transmission at the satellite and time of reception at the receiver.

7.8 Pseudorange Computations

Precise estimation of the pseudorange from a satellite to the receiver is crucial for a modern C/A code GPS receiver. The relationship between the standard deviation of the observations and that of the coordinates of the receiver position (see page 131) is

$$\sigma_{\text{pos}} = \sqrt{\sigma_e^2 + \sigma_n^2 + \sigma_u^2} = \text{PDOP}\ \sigma_0, \tag{7.31}$$

where σ_{pos} is the standard deviation of the receiver position, σ_0 is the standard deviation of unit weight. PDOP is the position dilution of precision, which depends on the geometry of the satellite constellation. Optimal accuracy of the position is obtained when the standard deviation of unit weight is as small as possible.

A pseudorange measurement is computed as the travel time from the satellite to the receiver multiplied by the speed of light in vacuum. The receiver has to estimate exactly when the start of a frame arrives at the receiver. This is done by adding the code phase to the time when the frame entered the receiver.

In Figure 7.21, the satellite transmits the start of the C/A code at $t = 0$ ms. This signal is received by the receiver approximately 70 ms after it is transmitted from the satellite. A range from the satellite to the receiver of 21,000 km corresponds to a travel time of 70 ms. As described before, the receiver is using block processing. That is, to calculate an accurate pseudorange and hereby an accurate position, the exact start of the C/A code in frame 71 in Figure 7.21 has to be found.

Figure 7.22 shows the first 700 samples of frame 71 in detail. The receiver has a time tag for the start of the frame. The problem is then to determine exactly where the start of the code is in the frame of data. In Figure 7.22, the start of the C/A code is at sample number 605, which is the correct code start in the simulated data used in this section.

Since the sampling frequency of the reference data set is at 38.192 MHz, each sample corresponds to

$$\Delta p = \frac{c}{38.192\,\text{MHz}} = 7.86\,\text{m}.$$

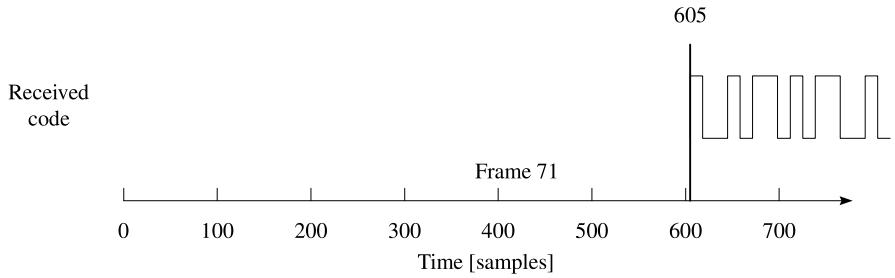


FIGURE 7.22. Close-up of frame 71 from Figure 7.21. In this case, the beginning of the C/A code is at sample 605 of the total of 38,192.

Since the prompt code is precisely aligned with the incoming signal to the nearest sample, the maximum error as a result of the discrete samples will be half that or < 5 m, which is sufficient for C/A code-type signals. If the sampling frequency is lower, a higher precision is required. Then it is possible to use the residual code phase at the end of each ms period to further refine that accuracy.